

Universidade de Vigo

Escola Superior de Enxeñaría Informática

Memoria do Traballo de Fin de Grao que presenta

D. Mauro Zelenka Pedrosa

para a obtención do Título de Graduado en Enxeñaría Informática

**Orquestación de Agentes Inteligentes mediante
Firebase AI Logic y Gemini para la Optimización
del Entrenamiento Físico en Android**

Xullo, 2026

Traballo de Fin de Grao Nº: EI 25/26-24

Titora: Eva M^a Lorenzo Iglesias

Cotitor: Pedro Celard Pérez

Área de coñecemento: Linguaxes e Sistemas Informáticos

Departamento: Informática



A mi familia, por su apoyo incondicional y por enseñarme que con esfuerzo todo es posible.

A mis tutores, Pedro y Eva, por su orientación y por la confianza depositada en mi trabajo.

Y a todos los que, de alguna manera, me motivaron a seguir creciendo como ingeniero.

Resumen

La gestión tradicional del entrenamiento físico mediante aplicaciones móviles suele limitarse a herramientas de registro pasivo que dependen de la interpretación manual del usuario. Este Trabajo de Fin de Grado presenta GymAI, una solución disruptiva que evoluciona el concepto de asistente deportivo hacia un ecosistema de agentes cognitivos orquestados mediante la infraestructura de Firebase AI Logic y el modelo fundacional Gemini. A diferencia de las soluciones convencionales, GymAI implementa procesos de razonamiento autónomos capaces de analizar el contexto biológico (vía *Health Connect*) y el historial de entrenamiento para generar estrategias de planificación proactivas. Desarrollada de forma nativa para Android bajo los principios de Clean Architecture, la aplicación integra persistencia *offline-first*, sincronización en la nube mediante *Firebase* y una interfaz adaptativa en Jetpack Compose. El resultado es una plataforma de alto rendimiento que no solo monitoriza la actividad, sino que actúa como un estratega inteligente para la optimización del rendimiento físico.

Abstract

Conventional fitness tracking applications often function as static data repositories, requiring significant user effort for meaningful interpretation. This project introduces GymAI, a next-generation Android ecosystem that pivots towards autonomous agentic orchestration. By leveraging the Firebase AI Logic architecture and the Gemini model, the system transcends simple chat interfaces to implement a multi-agent architecture capable of strategic reasoning.

GymAI autonomously interprets biological context through *Google Health Connect* and historical training data to formulate proactive workout strategies. Engineered with a robust Clean Architecture and an *offline-first* model with *Firebase* synchronization, the platform ensures scalability and responsiveness. The integration of Jetpack Compose provides a premium, adaptive user experience. Ultimately, this project demonstrates how merging specialized AI agents with modern mobile development can transform personal health management from passive recording to intelligent, pro-active guidance.

Índice General

1. Introducción	1
1.1. Motivación y Problemática	1
1.2. Contexto Tecnológico	2
1.3. Objetivos y Propuesta de Valor	2
2. Objetivos	3
2.1. De la Visión a la Realidad Técnica	3
2.2. Enfoque Estratégico	4
2.3. Objetivos Generales	4
2.4. Objetivos Específicos	5
2.5. Impacto del Proyecto	5
3. Resumen de la Solución Propuesta	6
3.1. Descripción General	6
3.2. Dinámica de Uso y Asistencia Inteligente	6
3.3. Áreas Funcionales de la Aplicación	7
3.4. Ecosistema Funcional de GymAI	8
3.5. Integración de Fuentes de Datos Externas	8
3.6. Disponibilidad y Funcionamiento sin Conexión	8
3.7. Resumen de la Metodología Empleada	9
4. Planificación y Metodología	10
4.1. Metodología de Trabajo: Scrum y Ciclos de Vida	10
4.2. Cronograma de Desarrollo (Gantt)	11

4.3. Seguimiento y Desviaciones Técnicas	12
5. Arquitectura del Sistema	14
5.1. Patrón de Presentación MVVM	14
5.2. Capas del Sistema (Clean Architecture)	15
5.2.1. Capa de Datos (Data Layer)	15
5.2.2. Capa de Dominio (Domain Layer)	15
5.2.3. Capa de Presentación (UI Layer)	15
5.3. Inyección de Dependencias con Hilt	16
5.4. Arquitectura de Capas	16
5.5. Flujos de Datos y Secuencia	17
5.6. Flujo de Comunicación y Navegación	17
6. Tecnologías y Herramientas	18
6.1. Lenguaje de Programación: Kotlin	19
6.2. Interfaz de Usuario y Diseño Moderno	19
6.3. Ecosistema de Inteligencia Artificial	19
6.4. Infraestructura y Desarrollo Asistido	20
7. Análisis de Requisitos	21
7.1. Matriz de Trazabilidad de Requisitos (RTM)	21
7.2. Historias de Usuario (User Stories)	22
7.3. Detalle de Requisitos y Criterios de Aceptación	22
7.4. Requisitos No Funcionales (RNF)	24
7.5. Diagrama de Casos de Uso	24
8. Diseño de Interfaz y UX	25
8.1. Estilo Visual: Material Design 3 y Material You	25
8.2. Metodología de Prototipado y Flujo de Trabajo	25
8.3. Tipografía e Iconografía Técnica	26
8.4. Principios de Experiencia de Usuario (UX) y Adaptabilidad	26

8.5. Galería de Pantallas	26
8.6. Arquitectura de Navegación y Flujo de Usuario	28
9. Gestión de Datos y Persistencia	29
9.1. Arquitectura de Persistencia Local (Room)	29
9.2. Diseño Lógico: Modelo Entidad-Relación (MER) y Paso a Tablas	30
9.3. Interoperabilidad y Transformación de Datos	31
9.4. Seguridad y Persistencia Adicional	31
9.5. Estrategia de Sincronización Offline-First	32
10. Validación y Pruebas	33
11. Manual de Usuario	34
11.1. Requisitos Mínimos del Sistema	34
11.2. Manual de Instalación	34
11.3. Guía de Navegación y Utilización	35
11.4. Inicio y Autenticación	35
11.5. Interacción con el Agent Orchestrator	35
11.6. Ciclo de Entrenamiento y Socialización	35
11.7. Resiliencia y Modo Offline	36
12. Principales Aportaciones	37
12.1. Orquestación de Agentes como Paradigma de Asistencia	37
12.2. Arquitectura Escalable y Reactiva	38
12.3. Interoperabilidad Total vía Health Connect	38
12.4. Integración con Firebase AI Logic	38
13. Conclusiones	39
14. Líneas de Trabajo Futuro	40
14.1. Soporte para Wear OS y Feedback Háptico	40
14.2. Visión por Computador para Corrección Técnica	40

14.3. Inteligencia Nutricional Multimodal	41
14.4. Privacidad y "On-Device AI" con Gemini Nano y Gemma	41
14.5. Sincronización con Gimnasios Inteligentes (IoT)	41
14.6. Orquestación de Enjambres de Agentes (Agent Swarms)	42
14.7. Gamificación y Ligas Comunitarias	42
15. Referencias	43
A. Anexos	45
A.1. Viabilidad Económica	45
A.2. Resumen Estratégico: Lean Canvas	46
A.3. Capturas del Prototipo en Figma	46
A.4. Lista de Acrónimos	46
A.5. Glosario de Términos	47

Índice de Ilustraciones

2.1. Diagrama de Venn del enfoque estratégico	4
3.1. Mapa del ecosistema funcional de GymAI	8
3.2. Ciclo de desarrollo incremental y adaptativo de GymAI.	9
4.1. Ciclo de desarrollo ágil adaptado	10
4.2. Diagrama de Gantt del proyecto.	13
5.1. Diagrama de arquitectura en TikZ	14
5.2. Arquitectura Clean Offline-First	16
5.3. Flujo de interacción entre capas	17
5.4. Sincronización asíncrona	17
6.1. Stack tecnológico integral	18
7.1. Mapa de trazabilidad técnica	21
7.2. Diagrama de casos de uso	24
8.1. Pantalla de gestión de rutinas activas.	27
8.2. Interactividad con el ecosistema de agentes.	27
8.3. Mapa de navegación jerárquico de GymAI.	28
9.1. Diagrama Entidad-Relación (MER)	30
9.2. Paso a Tablas.	30
9.3. Lógica de flujo para la persistencia y sincronización de datos en GymAI.	32
12.1. Diagrama de flujo del sistema de orquestación de agentes.	37

Índice de Tablas

2.1. Matriz de Correspondencia entre Visión y Realidad Técnica.	3
6.1. Resumen del ecosistema tecnológico (Stack).	19
6.2. Comparativa técnica de modelos fundacionales de IA (Google Gemini).	20
7.1. Cronograma de ejecución y catálogo de Historias de Usuario (MVP).	22
A.1. Estimación de costes operativos para el mantenimiento de GymAI.	45
A.2. Lean Canvas del proyecto GymAI.	46

Introducción

El teléfono móvil se ha convertido en una herramienta esencial en el día a día. Con la llegada de la inteligencia artificial, su utilidad ha crecido aún más. En los últimos años, las aplicaciones de gimnasio y salud han evolucionado notablemente, proporcionando métricas cada vez más detalladas, como repeticiones, cargas, tiempos de descanso o progresión. Sin embargo, hasta ahora se han centrado casi exclusivamente en la recopilación y visualización de los datos, dejando en un segundo plano su interpretación.

1.1. Motivación y Problemática

En el ámbito del entrenamiento físico, el proceso sigue siendo, en esencia, reactivo y estático: el usuario registra lo que realiza, pero continúa siendo él quien debe decidir qué hacer después. Esta falta de dirección genera incertidumbre tanto en principiantes, que no saben cómo estructurar sus sesiones, como en usuarios experimentados, que a menudo se enfrentan al estancamiento por no saber cómo variar sus rutinas de forma efectiva.

La repetición constante de las mismas rutinas, a menudo provocada por la falta de conocimiento, conduce al estancamiento y, en última instancia, a una pérdida significativa de motivación. Aunque la figura del entrenador personal resuelve este problema al proporcionar una guía experta, estructurada y dinámica, su coste económico lo hace inaccesible para la mayoría de los usuarios. Es en este punto donde se hace evidente la pieza que falta. A pesar de los avances en el ámbito de la Inteligencia Artificial (IA), todavía no se dispone de sistemas que acompañen activamente al deportista en su progreso, de forma accesible. No basta con saber que se han levantado cien kilos; es necesario comprender si ese esfuerzo es adecuado o si existe riesgo de sobreentrenamiento [1].

Este trabajo propone una aplicación basada en agentes inteligentes orquestados, capaces de analizar el contexto e historial del usuario para generar recomendaciones de entrenamiento adaptadas. De este modo, el sistema actúa como un asistente inteligente integrado en el propio dispositivo.

Se trata de dar el salto desde una herramienta de registro convencional, donde el usuario es quien recopila los datos, hacia un sistema interactivo en el que puede consultar, interpretar y utilizar esa información a través de un asistente basado en **IA** que le ayuda a crear y adaptar sus rutinas de entrenamiento. Esta transformación constituye el núcleo de este trabajo. A diferencia de enfoques completamente automatizados, el sistema mantiene al usuario en el centro del proceso, permitiéndole tomar decisiones informadas sobre su entrenamiento.

1.2. Contexto Tecnológico

Bajo este escenario, el desarrollo de GymAI aprovecha el potencial de la **IA** generativa. La integración de Modelos de Lenguaje de Gran Escala (**LLM**), como **Gemini** de Google [2], permite dotar a la aplicación de capacidades avanzadas de procesamiento para transformar datos en información útil para la toma de decisiones.

No se trata de algoritmos autónomos de aprendizaje evolutivo, sino de agentes de software que consultan múltiples fuentes de datos de forma orquestada para generar respuestas inteligentes bajo demanda.

Asimismo, el uso de estándares de interoperabilidad como **Health Connect** [3] garantiza que el sistema disponga de un contexto biológico del usuario. GymAI centraliza información de actividad física diaria, como el recuento de pasos, permitiendo que la **IA** analice el estado real del usuario antes de generar recomendaciones personalizadas. Estas funcionalidades de inteligencia avanzada operan bajo demanda y requieren conectividad a la red para su ejecución.

1.3. Objetivos y Propuesta de Valor

Este proyecto propone el desarrollo de una aplicación nativa para Android utilizando **Kotlin** y **Jetpack Compose**. El sistema se ha diseñado siguiendo un enfoque **Offline-First**, de modo que el usuario puede ejecutar y registrar sus entrenamientos sin depender de la conectividad, sincronizando los datos posteriormente.

Sobre esta base, se integran agentes de Inteligencia Artificial mediante la infraestructura de **Firestore AI Logic**, encargados de analizar la información disponible y generar recomendaciones personalizadas.

El objetivo central es el desarrollo de **GymAI**, un ecosistema que utiliza la orquestación de agentes de IA para ayudar al usuario a generar, adaptar y comprender sus rutinas de entrenamiento a partir de sus métricas de actividad y salud.

Objetivos

El objetivo central de este Trabajo de Fin de Grado (TFG) es el diseño y desarrollo de una aplicación móvil nativa para Android (GymAI) orientada a la asistencia en la gestión del entrenamiento físico. El proyecto se fundamenta en la orquestación de agentes de Inteligencia Artificial Generativa (GENAI), siendo capaz de interpretar métricas de actividad diaria para generar estrategias de entrenamiento dinámicas que se ajustan al estado fisiológico del deportista.

2.1. De la Visión a la Realidad Técnica

La trayectoria de este proyecto ha sido un proceso de aprendizaje y evolución constante. Lo que comenzó como una herramienta de registro, evolucionó hasta convertirse en un sistema capaz de interpretar el contexto del usuario.

En este camino, se han superado retos de ingeniería importantes: desde conectar a la comunidad en tiempo real hasta lograr que la IA traduzca el lenguaje natural en estructuras de datos (JavaScript Object Notation (JSON)).

El sistema utiliza los datos de salud (Health Connect) para asistir al usuario en sus decisiones de entrenamiento, combinando robustez técnica con un funcionamiento intuitivo. En el Cuadro 2.1 se resumen los pilares que sustentan esta propuesta.

Cuadro 2.1: Matriz de Correspondencia entre Visión y Realidad Técnica.

Visión (Objetivo)	Implementación (Realidad)	Impacto Técnico
Asistencia Inteligente	Orquestación de Agentes Gemini	Razonamiento contextual
Contexto Biológico	Google Health Connect	Integración de biometría real
Disponibilidad Total	Estrategia <i>Offline-First</i>	Persistencia de datos en Room
Arquitectura Escalable	Clean Architecture + MVVM	Código mantenible y desacoplado
Interacción Natural	Proxy de Procesamiento de NLP	Interfaz de chat integrada

2.2. Enfoque Estratégico

GymAI se sitúa en la convergencia de tres grandes áreas tecnológicas de la informática actual. El éxito del proyecto depende de la integración armónica de la Inteligencia Artificial, la Ingeniería de Software moderna y la interoperabilidad de datos de salud (ver Figura 2.1).

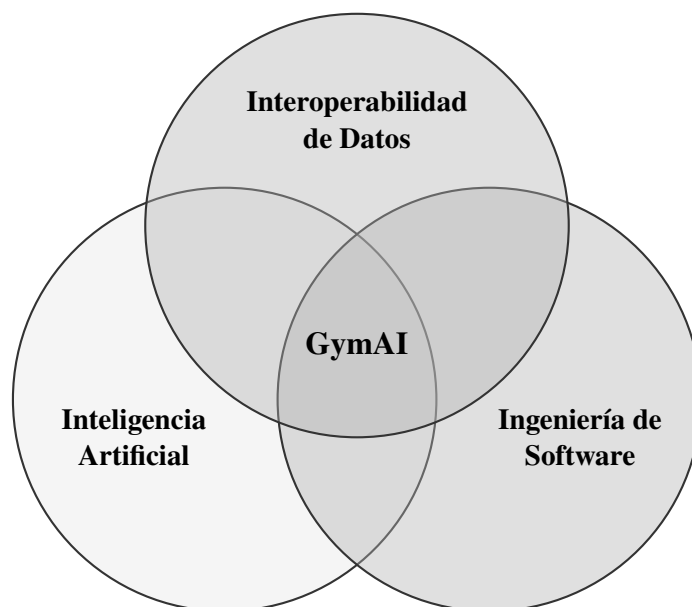


Figura 2.1: Diagrama de Venn del enfoque estratégico: Convergencia tecnológica de GymAI.

2.3. Objetivos Generales

Para dar cumplimiento a la propuesta, se han definido los siguientes hitos estratégicos que guían el desarrollo de la solución:

1. **Ecosistema de Agentes Inteligentes:** Implementar una arquitectura cognitiva basada en la infraestructura oficial de **Firestore AI Logic** (Google), permitiendo que la aplicación proporcione respuestas inteligentes a las peticiones del usuario. El objetivo es que la **IA** no se limite a un sistema de consulta estático, sino que actúe como un asistente que utilice el contexto disponible para asistir al usuario en la toma de decisiones.
2. **Arquitectura Escalable y Adaptativa:** Diseñar una base de código bajo los principios de *Clean Architecture* [4] y Modelo-Vista-Vista Modelo (**MVVM**) [5]. Esto facilita que el sistema evolucione y crezca en complejidad sin necesidad de reescrituras completas.
3. **Interoperabilidad de Datos de Salud:** Centralizar el contexto del usuario mediante **Health Connect**, eliminando los "silos de datos" entre aplicaciones de salud y entrenamiento que actualmente no se comunican entre sí.

2.4. Objetivos Específicos

De cara a alcanzar el objetivo principal, se desglosan las siguientes metas técnicas:

- **Persistencia Offline-First con Room:** El gimnasio es a menudo un entorno con falta de conexión a internet. Un objetivo clave es desarrollar un sistema que garantice la ejecución completa del entrenamiento sin interrupciones, sincronizando los resultados en segundo plano cuando la red esté disponible.
- **Orquestación con Gemini y Firebase:** Integrar el modelo Gemini [2] para potenciar agentes especializados (Estratega y Analista). Estos agentes deben ser capaces de interpretar el historial de actividad para generar planes de acción dinámicos.
- **Diseño Material You y Accesibilidad:** Aplicar las directrices de *Material Design 3* mediante **Jetpack Compose**. El objetivo es lograr una interfaz de alta calidad que se adapte al usuario, priorizando la legibilidad y la facilidad de interacción durante el entrenamiento.
- **Calidad y Mantenibilidad del Software:** Asegurar la estabilidad mediante el uso de **Hilt** [6] para la inyección de dependencias y la implementación de pruebas unitarias [7]. La robustez técnica es lo que permite que un proyecto académico se convierta en una aplicación de nivel profesional.
- **Factor Social y Motivacional:** Incorporar un feed social que permita compartir entrenamientos y planes, fomentando una experiencia colaborativa.

2.5. Impacto del Proyecto

Al alcanzar estos objetivos, GymAI se posicionará como una referencia de la potencialidad de la IA aplicada al desarrollo de software, demostrando cómo estas tecnologías pueden facilitar la vida de las personas y ahorrarles tiempo y esfuerzo.

La meta final de este trabajo es demostrar que, cuando la tecnología se diseña pensando en las personas y se apoya en la inteligencia artificial, se obtiene una herramienta que puede contribuir a mejorar el rendimiento deportivo y la salud del usuario mediante el uso de IA.

Resumen de la Solución Propuesta

Con el objetivo de transformar la gestión del rendimiento físico, se ha desarrollado GymAI. Se trata de un ecosistema móvil para **Android** que fusiona la potencia de la **IA** generativa junto con el análisis de datos de salud en tiempo real, permitiendo que la tecnología actúe como un compañero de entrenamiento inteligente y proactivo.

Para asegurar que la aplicación sea fiable y fácil de mejorar en el futuro, se ha construido siguiendo una estructura organizada por capas [4]. Este diseño profesional permite que el sistema sea escalable y crezca de forma ordenada, garantizando que GymAI pueda integrar nuevas funciones sin comprometer la calidad ni la estabilidad para el usuario.

3.1. Descripción General

Aunque en esencia GymAI funciona como una herramienta sólida para el registro de entrenamientos, su valor reside en la potencia de su asistente inteligente.

Este componente permite al deportista crear o modificar sus rutinas mediante **IA** de forma dinámica, adaptando el plan a sus necesidades del momento. Además, la aplicación no se limita a los registros numéricos, sino que puede leer datos biométricos (vía **Health Connect** [3]) para proporcionar al usuario un contexto real de su salud mientras progresa.

3.2. Dinámica de Uso y Asistencia Inteligente

La filosofía de GymAI se basa en dar libertad total al deportista. La aplicación funciona, ante todo, como un diario donde se pueden anotar pesos y ejercicios de forma manual. Sin embargo, la gran ventaja es que este proceso se puede hacer compartido con la IA: el usuario puede elegir entre crear y modificar sus propias rutinas o pedirle al asistente que lo haga por él si no sabe cómo ajustar el entrenamiento ese día.

Lo que hace especial a este asistente es su capacidad para conectar datos. Por un lado, la **IA** puede “leer” la base de datos local del móvil (**Room** [8]) para saber qué ha entrenado el usuario últimamente. Por otro, tiene permiso para consultar métricas como los pasos diarios a través de **Health Connect**. Esto permite que, si se le pregunta algo, el sistema no responda con frases genéricas, sino que use información real para dar una respuesta útil.

En conjunto, la interacción en GymAI busca que la tecnología no sea una barrera. El usuario sigue teniendo su cuaderno de entrenamiento de siempre, pero con un asistente que entiende tanto su historial de pesas como su actividad física diaria, facilitando que el deportista tome mejores decisiones sobre su salud sin complicaciones innecesarias.

3.3. Áreas Funcionales de la Aplicación

La experiencia de GymAI se organiza en cuatro bloques principales, cada uno diseñado para cumplir una función específica:

- **Gestor de Rutinas:** Es el corazón de la aplicación. Aquí es donde se crean y visualizan las rutinas. Se pueden consultar los ejercicios de cada rutina, se pueden registrar los pesos y repeticiones en tiempo real y el tiempo de entrenamiento durante la sesión.
- **Asistente Inteligente (GymAI Agents):** Pantalla dedicada a la interacción con la **IA**. Este módulo orquesta dos agentes especializados: el **Analista**, encargado de interpretar el historial de entrenamientos y la biometría del usuario, y el **Estratega**, que diseña o modifica los planes de entrenamiento. Juntos, permiten al usuario pedir que se cree un plan nuevo, se ajuste uno existente o se obtengan explicaciones sobre su estado de salud.
- **Feed Social y Comunidad:** Esta sección permite compartir los resultados de los entrenamientos. Es el espacio donde el esfuerzo individual se hace público para recibir apoyo de otros usuarios y ver qué están entrenando los demás miembros de la comunidad.
- **Perfil de Usuario y Biometría:** Se divide en dos pantallas. Por un lado, el perfil del usuario, donde se muestra su perfil personal y por otro, su actividad diaria (*dashboard*) que muestra los datos recuperados de **Health Connect**. Esta información sirve de base para que el asistente inteligente pueda ofrecer consejos personalizados.

3.4. Ecosistema Funcional de GymAI

La robustez de GymAI reside en su capacidad para orquestar múltiples servicios y fuentes de información de forma transparente para el usuario. La interacción entre estos componentes bajo el núcleo de la aplicación se resume en el siguiente ecosistema (ver Figura 3.1).

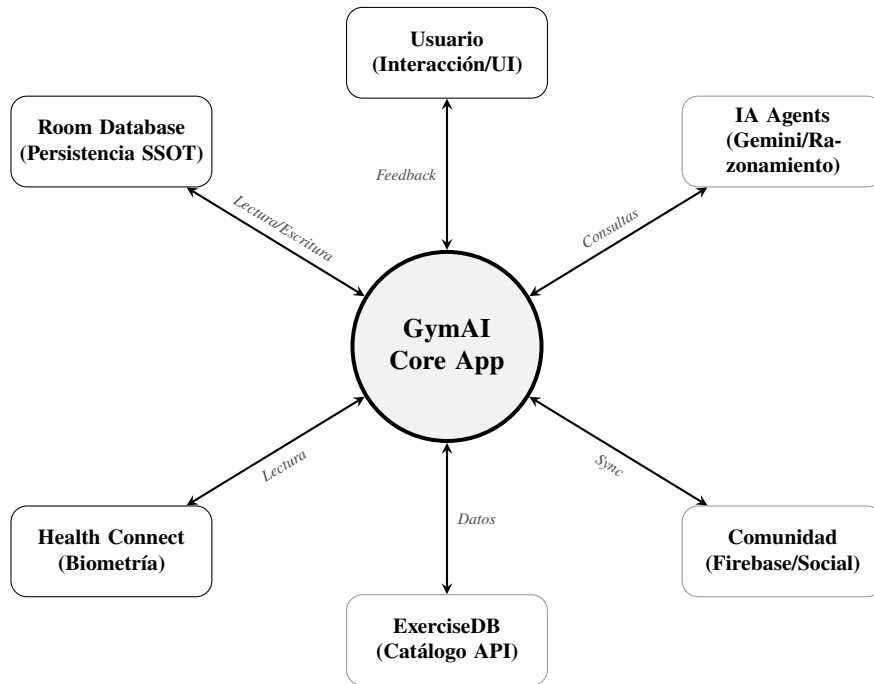


Figura 3.1: Mapa del ecosistema funcional de GymAI

3.5. Integración de Fuentes de Datos Externas

Para que GymAI sea útil desde el primer momento, integra la Interfaz de Programación de Aplicaciones (**API**) **ExerciseDB** [9], que proporciona una base de datos extensa de ejercicios con instrucciones detalladas e información visual. Esto permite disponer de un repositorio sólido sobre el cual construir las rutinas, tanto para el usuario como para los agentes inteligentes.

3.6. Disponibilidad y Funcionamiento sin Conexión

Uno de los principios fundamentales de GymAI es el enfoque **Offline-First** para las funciones esenciales, como el registro de entrenamientos y la consulta de rutinas. Esto garantiza que el usuario pueda completar su entrenamiento incluso sin conexión, sin riesgo de pérdida de datos.

No obstante, es importante señalar que las funciones más avanzadas, como el uso del asistente inteligente o la búsqueda de nuevos ejercicios en la nube, sí requieren conexión a internet. Este modelo híbrido permite ofrecer una aplicación muy fiable para el uso diario en el gimnasio, pero que a la vez puede aprovechar toda la potencia de la **IA** y la red cuando el usuario lo necesite.

3.7. Resumen de la Metodología Empleada

Para garantizar que la solución propuesta sea robusta y cumpla con los requisitos del usuario, el desarrollo se ha guiado por una metodología incremental e iterativa basada en principios **Agile**.

El proceso se ha estructurado en torno a tres pilares fundamentales que vinculan la técnica con la planificación (ver la figura 3.2):

- **Gestión por Historia de Usuarios (HUs):** Cada funcionalidad se ha definido desde la perspectiva del deportista, asegurando que el desarrollo técnico esté siempre alineado con una necesidad real y tangible.
- **Desarrollo Orientado a Arquitectura:** Se ha seguido estrictamente el patrón **Clean Architecture** [4], lo que ha permitido separar la lógica de negocio (agentes de **IA** y entrenamientos) de la infraestructura técnica, facilitando pruebas y cambios futuros.
- **Validación Continua:** Cada iteración se ha validado mediante pruebas en dispositivos reales, permitiendo ajustar el comportamiento de los agentes inteligentes y la interfaz de usuario de forma dinámica según los resultados obtenidos.

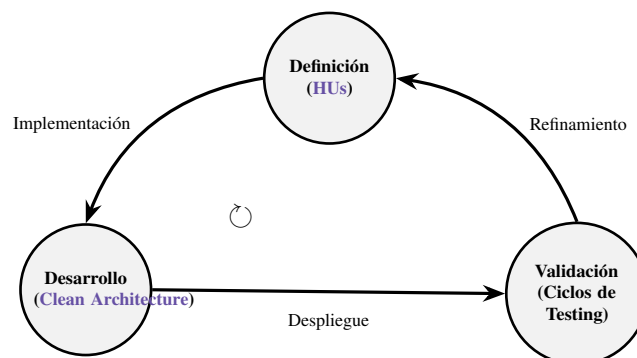


Figura 3.2: Ciclo de desarrollo incremental y adaptativo de GymAI.

Planificación y Metodología

Para pasar de la visión teórica a una aplicación que realmente ayuda al usuario, dividimos el camino en hitos prácticos. Esta planificación no es solo un calendario, sino el reflejo de cómo GymAI fue creciendo pieza a pieza, priorizando siempre que el sistema fuera robusto antes de añadir nuevas capas.

4.1. Metodología de Trabajo: Scrum y Ciclos de Vida

Para garantizar la calidad y adaptabilidad del software, se ha implementado un modelo de desarrollo **Agile** basado en ciclos de vida **IA** iterativos e incrementales (ver Figura 4.1).

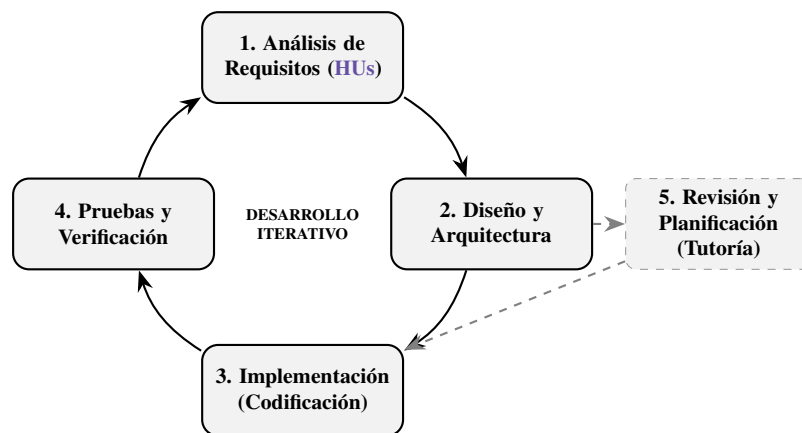


Figura 4.1: Ciclo de desarrollo ágil adaptado: Iteraciones incrementales con supervisión técnica.

Se ha utilizado una adaptación de la metodología **Agile**, inspirada específicamente en el marco de trabajo **Scrum** [10]. El flujo de trabajo se dividió en bloques fijos de dos semanas denominados **Sprint**, estructurados de la siguiente manera:

1. **Planificación del Sprint:** Definición de tareas críticas extraídas del **Product Backlog** para completar el incremento funcional del ciclo.
2. **Desarrollo Iterativo:** Implementación de funcionalidades siguiendo los principios de **Clean Architecture**.

3. **Revisión del Sprint:** Evaluación del incremento funcional antes de su integración definitiva en la rama principal.

Para la orquestación técnica, se siguió un modelo de desarrollo basado en ramas (**Feature Branching**) en **GitHub**. Cada nueva funcionalidad o corrección de error se desarrolló en una rama independiente, integrándose en la rama `main` únicamente tras superar las pruebas automatizadas y una revisión manual, garantizando así la estabilidad continua del proyecto móvil.

4.2. Cronograma de Desarrollo (Gantt)

El proyecto se desarrolló en un periodo total de aproximadamente 11 meses (44 semanas), estructurado cronológicamente para mitigar riesgos técnicos. A continuación se describen las fases principales que se reflejan en el diagrama de Gantt (ver Figura 4.2):

- **Fase 0: Concepción y Viabilidad:** Definición de la idea central, envío de propuesta técnica e investigación de la viabilidad de agentes de **IA** aplicada en dispositivos móviles.
- **Fase 1: Setup Inicial y PoC:** Configuración del entorno de desarrollo (**Gradle**, **Clean Architecture** [4], **Hilt** [6]) y creación de una prueba de concepto para la base de datos.
- **Fase 2: Requisitos y Diseño:** Elaboración de historias de usuario, diseño de Modelo Entidad-Relación (**MER**) y paso a tablas, diagramas de arquitectura y tecnologías, UML (Casos de Uso y Secuencia) y creación de prototipos de alta fidelidad usando **Figma** y **Google Stitch** [11], [12].
- **Fase 3: Refactorización y Persistencia:** Implementación final del esquema de **Room** [8], desarrollo de Data Access Object. Patrón que abstrae y encapsula todo el acceso a la fuente de datos (**DAOs**), conversores de tipos y configuración de **DataStore** [13].
- **Fase 4: Integración de Datos y Core Lógico:** Ingesta y mapeo del catálogo de ejercicios (**ExerciseDB** [9]), implementación de repositorios y casos de uso base.
- **Fase 5: Frontend con JetpackCompose:** Desarrollo del sistema de diseño, navegación y pantallas principales (Feed Social, Rutinas, Asistente Inteligente y Perfil/Biometría).
- **Fase 6: Inteligencia Artificial y Asistente:** Implementación del cliente de red para **Gemini** [2], diseño de estrategias de **Prompt Engineering** y funcionalidad de chat inteligente.
- **Fase 7: Cloud y Sincronización Social:** Integración con **Firebase Auth** [14] y **Firestore**, y desarrollo de tareas en segundo plano con **WorkManager** para la sincronización **Offline-First**.
- **Fase 8: Pruebas y Release:** Control de calidad mediante pruebas unitarias, corrección de errores críticos y generación de la Android Package Kit (**APK**) final para distribución.

- **Fase 9: Documentación y Defensa:** Redacción de la memoria técnica, análisis de viabilidad y síntesis de resultados para la defensa pública.

La distribución temporal de estas fases, las dependencias entre tareas y los hitos clave del proyecto se detallan en el cronograma de actividades (ver Figura 4.2), que proporciona una visión holística del esfuerzo invertido a lo largo del curso académico.

4.3. Seguimiento y Desviaciones Técnicas

A pesar de la planificación inicial, el proyecto sufrió varias desviaciones estratégicas que aumentaron significativamente su complejidad técnica. Lo que en la fase de concepción se planteó como una aplicación de fitness convencional, pivotó hacia un sistema social y posteriormente hacia un ecosistema de asistencia artificial.

Especialmente crítica fue la evolución de la capa de IA: se barajaron opciones iniciales como un simple **Chatbot** reactivo, para luego pasar a un sistema generador de rutinas mediante la interpretación de estructuras **JSON** generadas por la IA y procesadas con **Moshi** [15] para su persistencia en **Room** [8]. Finalmente, en el mes de abril, se tomó la decisión técnica de saltar hacia una arquitectura de agentes orquestados capaces de leer el contexto de **Health Connect** [3].

Esta trayectoria, aunque provocó una desviación de una semana en la Fase 4 debido a la complejidad de integrar la biometría y la necesidad de migrar de **Kapt** [16] a **KSP** [17], permitió cumplir con el 100 % de los nuevos requisitos funcionales de alto nivel, logrando un producto final mucho más robusto y adaptado al estado del arte tecnológico actual.

La distribución temporal de estas fases, las dependencias entre tareas y los hitos clave del proyecto se detallan en el cronograma de actividades (ver Figura 4.2), que proporciona una visión holística del esfuerzo invertido a lo largo del curso académico.

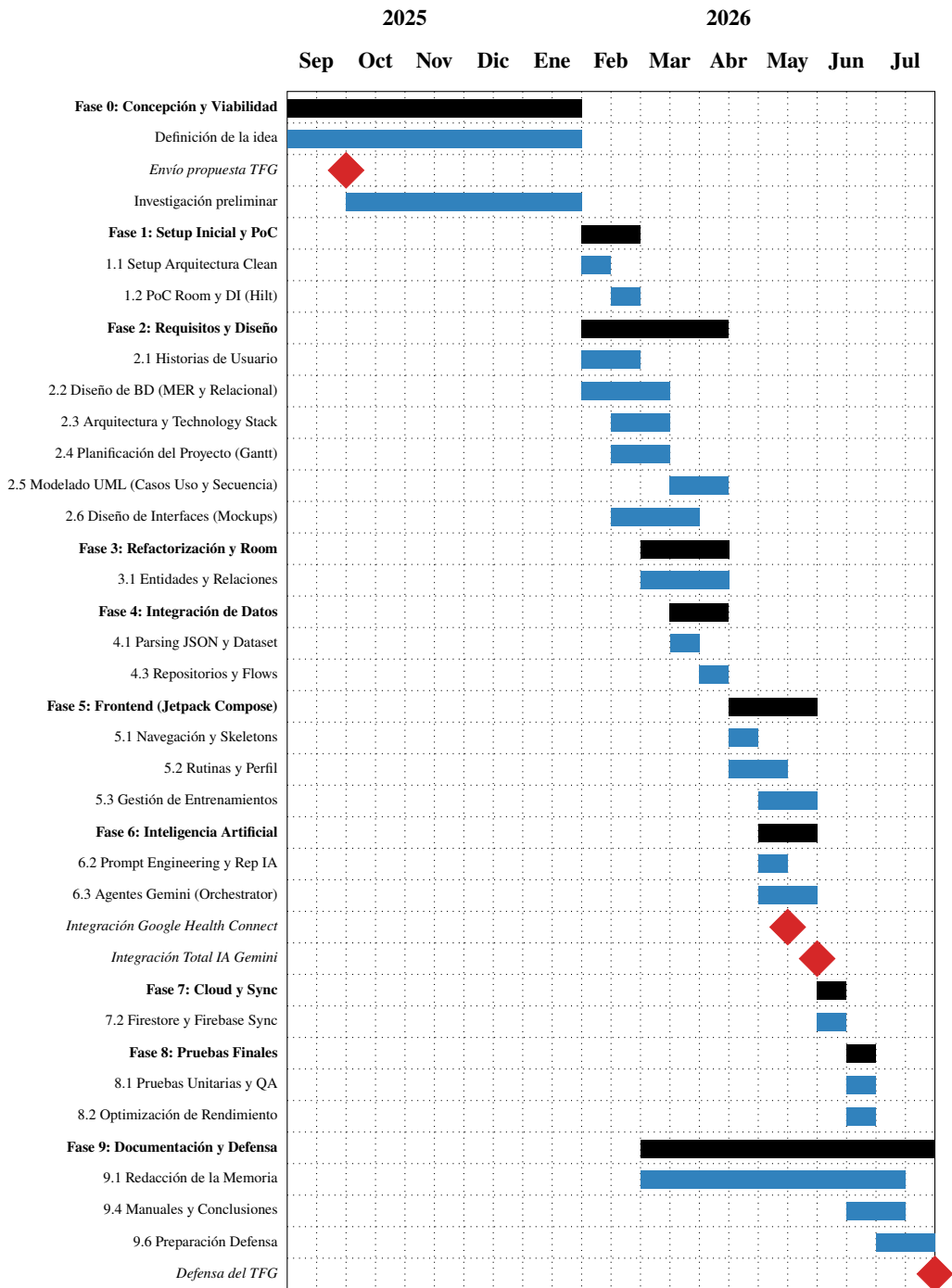


Figura 4.2: Diagrama de Gantt del proyecto.

Arquitectura del Sistema

La arquitectura de GymAI ha sido diseñada bajo los principios de **Clean Architecture** [4] y el patrón de presentación **MVVM** [5]. Este enfoque, detallado visualmente en la figura 5.1, garantiza una separación total de intereses, facilitando la mantenibilidad, la escalabilidad y la capacidad de realizar pruebas unitarias sobre la lógica de negocio.

Ventaja de la Clean Architecture

El uso de capas concéntricas permite que los cambios en los detalles de implementación (como sustituir la base de datos **Room** por una **API** externa o cambiar el modelo de **IA**) no afecten a la lógica de negocio central, protegiendo el código más valioso del sistema.

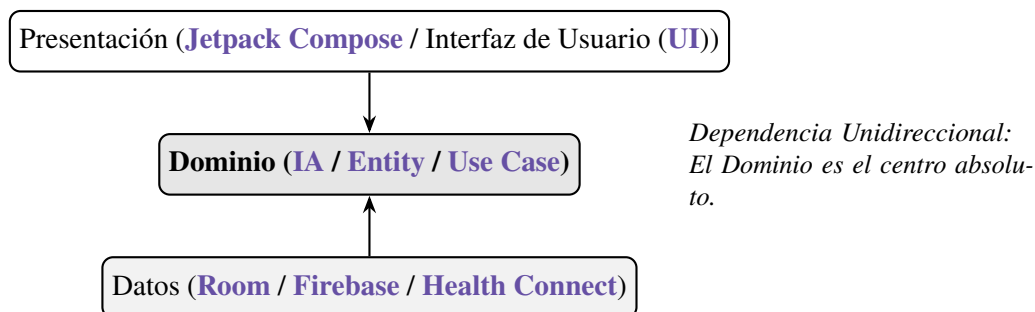


Figura 5.1: Diagrama de arquitectura por capas vectorizado en TikZ.

5.1. Patrón de Presentación MVVM

El núcleo de la interfaz de usuario se desacopla de la lógica de negocio mediante este patrón fundamental:

- **View:** Compuesta por funciones componibles de **Jetpack Compose** que observan el estado de la Interfaz de Usuario (**UI**) y notifican eventos de usuario al **ViewModel**. Carecen de toda lógica de decisión.

- **ViewModel:** Actúa como puente entre la **UI** y el Dominio. Almacena el estado de la pantalla en un objeto **StateFlow** y gestiona la ejecución de corrutinas para la carga de datos.
- **UIState:** Un objeto inmutable que representa la "fotografía" de la pantalla en cada momento, garantizando que la **UI** sea una función pura del estado.

5.2. Capas del Sistema (Clean Architecture)

El proyecto se organiza en tres capas concéntricas donde las dependencias siempre apuntan hacia el interior:

5.2.1. Capa de Datos (Data Layer)

Responsable de gestionar las fuentes de información (Fuente Única de Verdad (**SSOT**)). Implementa el patrón **Repository** para decidir de forma transparente el origen de los datos:

- **Persistencia Local (Room):** Objetos de acceso a datos (**DAOs**) que interactúan con **SQLite** para garantizar la funcionalidad *offline*.
- **Red y Agentes externos:** Servicios que consumen la **API** de **Gemini** y **ExerciseDB**.
- **External Buffers:** Integración con **Firestore** para la sincronización comunitaria y **Health Connect** para métricas biométricas.

5.2.2. Capa de Dominio (Domain Layer)

Es la capa más crítica e independiente del *framework* de **Android**. Contiene la "verdad" del proyecto:

- **Orquestación de Agentes (Firebase AI):** Utiliza el motor de razonamiento de **Gemini** para orquestar agentes mediante **Function Calling**. Los agentes actúan como mediadores inteligentes que interactúan con el hardware y las **APIs** del dispositivo.
- **UseCase:** Encapsulan una única acción de negocio (ej: `GenerateWorkoutProposalUseCase`).
- **Entity:** Modelos de datos puros que definen objetos como `TrainingPlan` o `HealthMetric`.

5.2.3. Capa de Presentación (UI Layer)

Capa encargada de la representación visual utilizando un enfoque declarativo. Está organizada por módulos funcionales que comparten componentes de diseño atómico a través de un módulo `core`.

5.3. Inyección de Dependencias con Hilt

Para gestionar el ciclo de vida de los objetos y evitar el acoplamiento fuerte, se ha utilizado **Dagger Hilt** [6]. Esta herramienta inyecta las dependencias necesarias de forma jerárquica.

```

1 @Module
2 @InstallIn(SingletonComponent::class)
3 object AgentModule {
4
5     @Provides
6     @Singleton
7     fun provideAgentOrchestrator(
8         analyst: FitnessAnalystAgent,
9         strategist: WorkoutStrategistAgent
10    ): AgentOrchestrator {
11         return AgentOrchestrator(analyst, strategist)
12     }
13 }

```

Listing 5.1: Ejemplo de módulo Hilt para la provisión del orquestador de agentes de IA.

5.4. Arquitectura de Capas

La aplicación sigue una arquitectura de tres capas basada en los principios de *Clean Architecture*, asegurando la separación de responsabilidades y facilitando la escalabilidad del sistema. A continuación se detalla la interacción entre las capas de Presentación, Dominio y Datos (ver Figura 5.2).

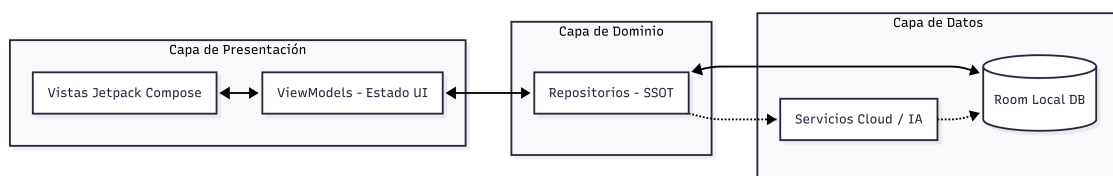


Figura 5.2: Arquitectura Clean Offline-First: Flujo de comunicación entre Presentación, Dominio y Datos.

5.5. Flujos de Datos y Secuencia

El comportamiento dinámico del sistema se ilustra mediante diagramas de secuencia que detallan la interacción entre las capas. En la figura 5.3 se analiza la generación de rutinas, mientras que la figura 5.4 describe el proceso de sincronización asíncrona:

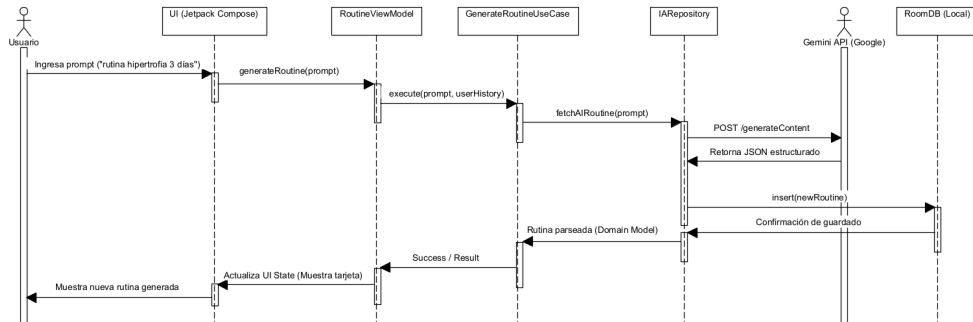


Figura 5.3: Flujo de interacción entre capas para la generación y ajuste inteligente mediante el ecosistema de agentes.

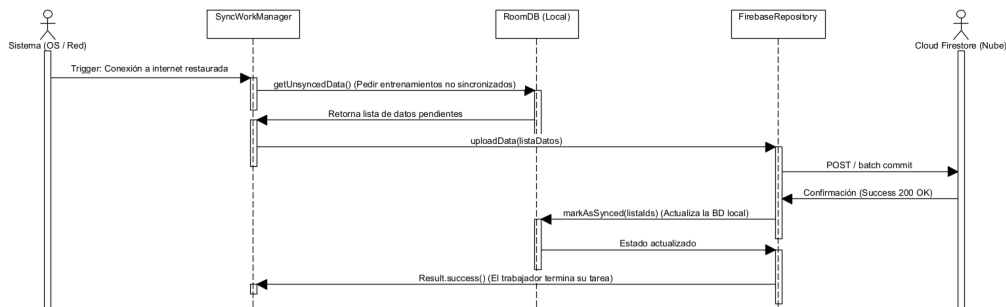


Figura 5.4: Proceso de sincronización asíncrona mediante WorkManager y persistencia en la nube.

5.6. Flujo de Comunicación y Navegación

La comunicación entre capas es estrictamente unidireccional. El uso de **Navigation Compose** [18] permite definir un grafo de navegación centralizado donde cada funcionalidad es un nodo independiente, minimizando los errores en tiempo de ejecución asociados a claves *string* manuales.

Tecnologías y Herramientas

Para el desarrollo de GymAI se seleccionó un conjunto de tecnologías de última generación que garantizan la eficiencia, la mantenibilidad y una excelente experiencia de usuario en el ecosistema **Android**, alineándose con las guías de Modern Android Development (**MAD**).

Modern Android Development (MAD)

MAD es el conjunto de herramientas (**Kotlin**, **Jetpack Compose**, **Hilt**) y guías que Google recomienda para crear aplicaciones de alta calidad [19]. Elimina el código redundante (*boilerplate*) y permite centrarse en la funcionalidad central del sistema.

Las herramientas seleccionadas y su disposición en la arquitectura general del proyecto se pueden visualizar en la figura 6.1.

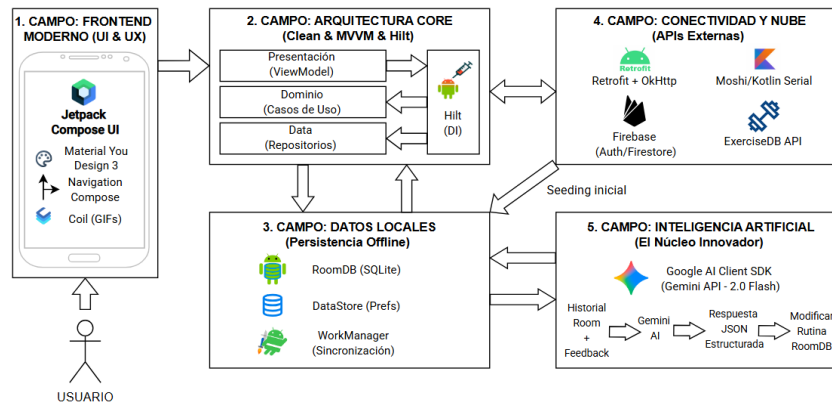


Figura 6.1: Stack tecnológico integral y ecosistema de herramientas del proyecto.

A nivel de componentes, este ecosistema tecnológico se detalla de forma resumida en el cuadro 6.1, donde se especifican las tecnologías concretas empleadas en cada capa del sistema.

Cuadro 6.1: Resumen del ecosistema tecnológico (Stack).

Capa / Categoría	Tecnología / Framework
Lenguaje	Kotlin (K2)
Interfaz de Usuario (UI)	Jetpack Compose (Material 3)
Arquitectura	MVVM + Clean Architecture
Inyección de Dependencias	Hilt (Dagger)
Base de Datos Local	Room (SQLite)
Preferencias de Usuario	DataStore
Red y Procesamiento	Retrofit + OkHttp + Moshi
Inteligencia Artificial	Gemini (v2.5 Flash)
Backend y Autenticación	Firebase (Firebase Auth y Firestore)
Tareas en Segundo Plano	WorkManager (Sync)

6.1. Lenguaje de Programación: Kotlin

Kotlin se utilizó como lenguaje principal de forma exclusiva. Su naturaleza multiparadigma y características como la *null-safety* estricta fueron fundamentales para reducir los errores en tiempo de ejecución. El proyecto utiliza el nuevo compilador **K2** y **Kotlin Coroutines** [20] para la gestión de procesos asíncronos en segundo plano, evitando el bloqueo del hilo principal de la interfaz (**UI**). El uso de **Flow** [21] permite la gestión reactiva de flujos de datos continuos.

6.2. Interfaz de Usuario y Diseño Moderno

- **JetpackCompose**: Se adoptó este framework declarativo para la construcción de la interfaz [18]. **Jetpack Compose** permite que la **UI** sea una función pura del estado, eliminando errores derivados de la desincronización entre el código y la vista.
- **Material3 (Material You)**: La aplicación implementa el sistema de colores dinámicos [22], adaptándose a la paleta del fondo de pantalla del usuario.
- **Ecosistema de Prototipado**: El diseño visual se orquestó mediante el uso de **Google Stitch** [12] para la generación rápida de mockups iniciales y **Figma** [11] para la definición detallada de atributos técnicos.
- **Coil**: Librería utilizada para la carga eficiente de imágenes y GIFs animados [23].

6.3. Ecosistema de Inteligencia Artificial

- **Gemini**: El motor de razonamiento principal [2]. Se utiliza para el procesamiento de lenguaje natural y la generación de rutinas adaptativas en formato **JSON**. En el cuadro 6.2 se presenta

una comparativa técnica que justifica la elección del modelo *Flash* frente al *Pro* para esta aplicación.

- **Firestore**: Infraestructura oficial de orquestación de agentes [24] que permite definir estrategias de planificación y memoria persistente.
- **Firebase**: Gestión de autenticación [14], analíticas y monitorización de IA mediante la consola del ecosistema.

Cuadro 6.2: Comparativa técnica de modelos fundacionales de IA (Google Gemini).

Atributo	Gemini 2.5 Pro	Gemini 2.5 Flash	Impacto en GymAI
Latencia	Media	Muy Baja	Respuesta casi instantánea
Ventana de Contexto	2M tokens	1M tokens	Suficiente para historial
Capacidad de Razonamiento	Alta	Media-Alta	Óptima para agentes
Coste por Token	Elevado	Económico	Viabilidad del proyecto

6.4. Infraestructura y Desarrollo Asistido

- **Hilt (Dagger)**: Framework oficial de inyección de dependencias [6] que automatiza el ciclo de vida de los contenedores y simplifica la modularización.
- **Antigravity**: Como parte del paradigma moderno de desarrollo, se utilizó el agente inteligente **Antigravity** [25]. Este asistente de IA integrado en el IDE facilitó la generación de código base, la refactorización de componentes complejos y la validación de patrones de arquitectura en tiempo real.
- **AndroidStudio**: El entorno de desarrollo integrado principal [26], facilitando la depuración e inspección de diseños de Compose en tiempo real.
- **Git y GitHub**: Herramientas fundamentales para el control de versiones [27].

Análisis de Requisitos

En esta fase inicial del proyecto se realizó una labor exhaustiva de toma de requisitos y análisis de necesidades para definir con precisión el alcance funcional de GymAI. Siguiendo metodologías ágiles, los requisitos se han estructurado en Historias de Usuario (HUs) que capturan el valor de negocio desde la perspectiva del deportista.

7.1. Matriz de Trazabilidad de Requisitos (RTM)

Para garantizar la integridad del sistema, la Matriz de Trazabilidad relaciona cada Historia de Usuario con el componente técnico responsable de su implementación y el método de validación empleado, tal como se puede observar en el figura 7.1.

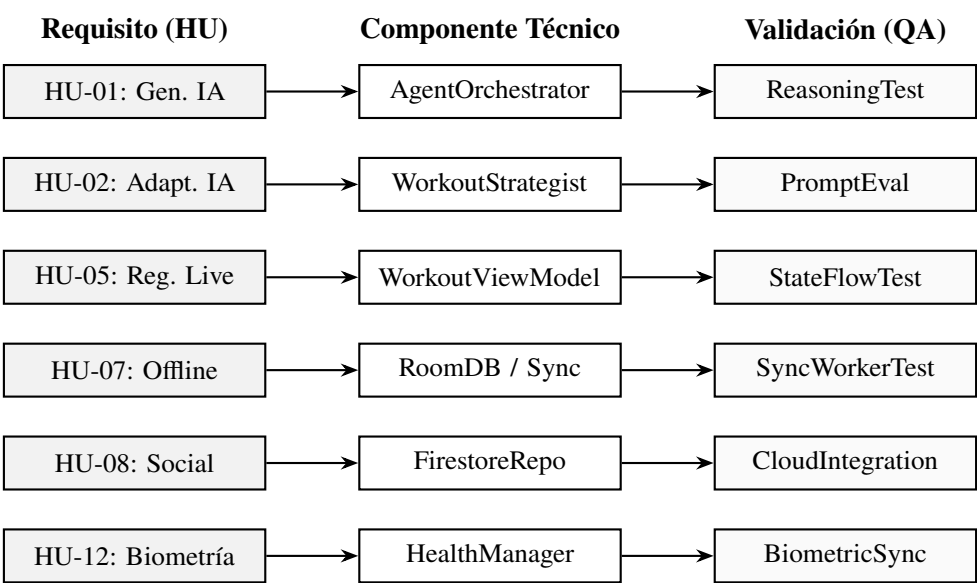


Figura 7.1: Mapa de trazabilidad técnica: Conectando requisitos funcionales con arquitectura y validación.

7.2. Historias de Usuario (User Stories)

A continuación se resumen las historias de usuario que han guiado el desarrollo del producto mínimo viable (MVP) de GymAI, detalladas en el cuadro 7.1.

Cuadro 7.1: Cronograma de ejecución y catálogo de Historias de Usuario (MVP).

Fase / Iteración	Fecha Inicio	Fecha Fin	Duración
Hito 1 (Fases 3-4): Persistencia y Dominio	09 mar 26	05 abr 26	Entrega 1
HU-07 Persistencia Offline (Room)	09 mar 26	20 mar 26	11 días
HU-03 Gestión Library (Catálogo)	21 mar 26	05 abr 26	15 días
Hito 2 (Fase 5): Rutinas y Seguimiento	06 abr 26	16 abr 26	Entrega 2
HU-04 Creación Manual	06 abr 26	08 abr 26	3 días
HU-05 Registro Live (Entrenamiento)	09 abr 26	11 abr 26	3 días
HU-11 Salud y Perfil	12 abr 26	14 abr 26	3 días
HU-12 Salud Pasiva (Health Connect)	15 abr 26	16 abr 26	2 días
Hito 3 (Fase 6): Inteligencia Artificial	17 abr 26	30 abr 26	Entrega 3
HU-01 Generación de Rutinas IA	17 abr 26	23 abr 26	7 días
HU-02 Adaptación IA (Agentes)	24 abr 26	30 abr 26	7 días
Hito 4 (Fase 7): Ecosistema Social y Cloud	01 may 26	31 may 26	Entrega 4
HU-06 Autenticación Firebase	01 may 26	05 may 26	5 días
HU-08 Publicación de Feed	06 may 26	15 may 26	10 días
HU-09 Social Comments	16 may 26	22 may 26	7 días
HU-10 Social Search	23 may 26	31 may 26	9 días

7.3. Detalle de Requisitos y Criterios de Aceptación

Cada historia de usuario se desglosa en criterios de aceptación técnicos que definen cuándo se considera finalizada una funcionalidad.

HU-01	Generación de rutinas personalizadas mediante IA
Descripción	Como usuario, quiero solicitar al asistente virtual que me cree una rutina indicando mis objetivos y nivel, para obtener un plan estructurado al instante.
Criterios de Aceptación	• El sistema interpreta el mensaje del usuario y devuelve un texto de respuesta. • Se muestra una tarjeta visual interactiva con el resumen de la rutina. • Se proporcionan opciones para guardar o empezar la rutina en el momento.

HU-02	Adaptación de ejercicios por lesiones o preferencias
Descripción	Como usuario, quiero informar al asistente si un ejercicio me causa molestias, para que la IA me ofrezca una alternativa segura.
Criterios de Aceptación	• El asistente recomienda un ejercicio alternativo del mismo grupo muscular. • La interfaz indica claramente el ejercicio eliminado y el sustituto.
HU-05	Registro de entrenamiento en tiempo real
Descripción	Como usuario, quiero marcar las series y repeticiones a medida que las completo durante la rutina activa.
Criterios de Aceptación	• Se visualiza un cronómetro activo y checkboxes para cada serie. • Permite modificar peso y repeticiones reales si varían del objetivo.
HU-07	Funcionamiento sin conexión (Offline-first)
Descripción	Como usuario, quiero registrar mis entrenamientos en el gimnasio aunque no tenga cobertura a internet.
Criterios de Aceptación	• Persistencia local inmediata en la base de datos Room del dispositivo. • Sincronización transparente con la nube mediante WorkManager al recuperar la red.
HU-08	Publicación de entrenamientos en el feed social
Descripción	Como usuario, quiero compartir mis entrenamientos con la comunidad para recibir apoyo de mis seguidores.
Criterios de Aceptación	• Permite crear un post con texto y adjuntar imágenes (vía Coil). • La publicación es visible cronológicamente en el feed de los seguidores.
HU-12	Integración Biométrica Pasiva (Health Connect)
Descripción	Como usuario, quiero que la aplicación lea mis pasos y calidad de sueño automáticamente para contextualizar mi entrenamiento.
Criterios de Aceptación	• Consentimiento granular del usuario para la lectura de datos de salud. • Importación periódica de biometría que alimenta el contexto enviado a Gemini.

Diseño de Interfaz y UX

El diseño de GymAI prioriza la visibilidad, la simplicidad y el impacto emocional, siguiendo los estándares modernos de diseño de aplicaciones Android para lograr una estética "premium" y funcional. La interfaz ha sido concebida no solo como un medio de entrada de datos, sino como un facilitador del progreso deportivo y la motivación del usuario.

8.1. Estilo Visual: Material Design 3 y Material You

Se ha adoptado el sistema **Material Design 3 (Material You)** [22], que permite una personalización profunda y el uso de colores dinámicos basados en el sistema operativo.

8.2. Metodología de Prototipado y Flujo de Trabajo

El proceso de diseño siguió un flujo de trabajo iterativo dividido en dos fases técnicas complementarias:

1. **Concepción Rápida (Google Stitch):** Se utilizó **Google Stitch** [12] como herramienta inicial de prototipado para materializar las guías de *Material Design 3*. Esta fase permitió validar de forma ágil la navegación principal y la disposición de los componentes atómicos antes de entrar en el detalle técnico.
2. **Definición de Atributos y Alta Fidelidad (Figma):** Una vez validados los mockups iniciales, se trasladaron a **Figma** [11]. En esta etapa se definieron con exactitud todos los atributos visuales: espaciados exactos (*padding*s), radios de curvatura de contenedores, jerarquía tipográfica fina y la exportación de *assets* gráficos (como los iconos personalizados y GIFs de ejercicios).

8.3. Tipografía e Iconografía Técnica

La jerarquía visual se establece mediante el uso de dos fuentes principales de *Google Fonts*:

- **Outfit**: Utilizada exclusivamente para titulares y métricas clave debido a su geometría moderna y alta legibilidad.
- **Inter**: Seleccionada para el cuerpo de texto y etiquetas técnicas por su excelente renderización en pantallas de alta densidad de píxeles (*DPI*).

8.4. Principios de Experiencia de Usuario (UX) y Adaptabilidad

- **Feedback Háptico y Sensorial**: La aplicación incorpora micro-animaciones de transición y vibraciones sutiles al completar las series de entrenamiento.
- **Categorización por Tarjetas (Cards)**: Se han utilizado componentes de tarjeta con bordes rounded y sombreados para separar módulos funcionales.
- **Diseño de "Pulgar"**: Los elementos interactivos más críticos se sitúan en la zona inferior de la pantalla para facilitar el uso del dispositivo con una sola mano.
- **Responsividad Nativa**: Gracias al uso de **Jetpack Compose** y *Window Size Classes*, la interfaz se adapta dinámicamente desde teléfonos compactos hasta tablets.

8.5. Galería de Pantallas

Como resultado del proceso de diseño y prototipado, se han obtenido las pantallas finales que componen la experiencia nuclear de la aplicación. En la figura 8.1 se puede observar la gestión de sesiones activas, mientras que en la figura 8.2 se detalla la interfaz de interacción con el ecosistema de agentes inteligentes:

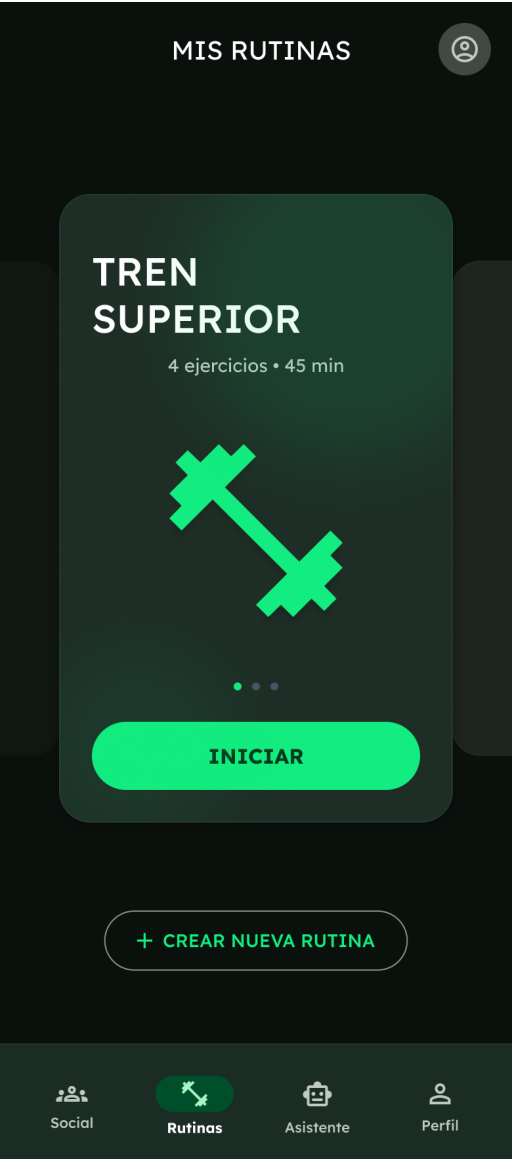


Figura 8.1: Pantalla de gestión de rutinas activas.



Figura 8.2: Interactividad con el ecosistema de agentes.

8.6. Arquitectura de Navegación y Flujo de Usuario

Esta interfaz se articula siguiendo un modelo jerárquico que facilita el acceso a las funciones más críticas mediante una barra de navegación inferior (*Bottom Bar*). Como se detalla en el mapa de navegación de la figura 8.3, el flujo de usuario se divide en tres niveles lógicos: un nivel de pre-autenticación, el nivel principal de módulos (Social, Rutinas, Asistente y Perfil), y un nivel de detalle para tareas específicas. Esta estructura garantiza que el usuario pueda conmutar entre los pilares de la aplicación con un solo toque, manteniendo la coherencia de estado entre pantallas.

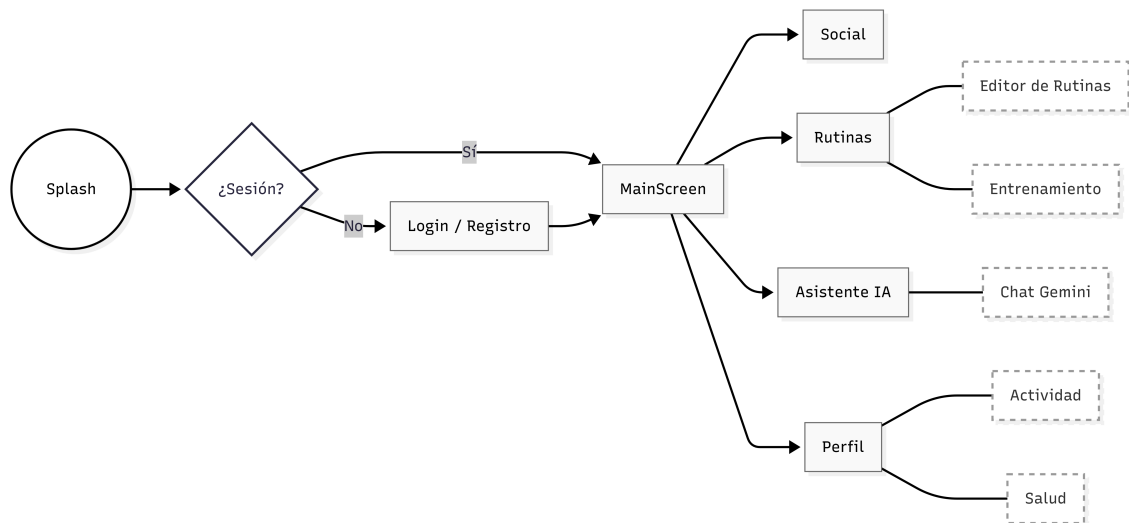


Figura 8.3: Mapa de navegación jerárquico de GymAI.

Gestión de Datos y Persistencia

La gestión de la información es el pilar crítico de GymAI, ya que el sistema debe orquestar datos estructurados de entrenamiento, métricas biométricas sensibles y el estado comunitario en tiempo real, garantizando siempre la integridad y disponibilidad *offline*.

9.1. Arquitectura de Persistencia Local (Room)

Para la persistencia en el dispositivo se diseñó una base de datos relacional robusta utilizando **Room Persistence Library** [8]. Esta elección no solo proporciona una capa de abstracción sobre SQLite, sino que permite la verificación de esquemas en tiempo de compilación, minimizando errores en producción.

Las entidades principales que articulan el sistema son:

- **UserEntity**: Almacena el perfil del usuario, objetivos biométricos y metadatos de sincronización regional.
- **RoutineEntity** y **ExerciseEntity**: Forman la columna vertebral del catálogo. Se relacionan mediante una tabla de unión (**RoutineExerciseCrossRef**) que permite que un ejercicio pertenezca a múltiples rutinas con un orden y volumen específicos.
- **TrainingSessionEntity**: Registra cada entrenamiento finalizado, vinculando las series realizadas con las cargas y repeticiones reales.
- **HealthCacheEntity**: Almacena de forma temporal las métricas obtenidas de **Health Connect** [3] para reducir la latencia de acceso a la API y permitir el razonamiento de los agentes en modo offline.

9.2. Diseño Lógico: Modelo Entidad-Relación (MER) y Paso a Tablas

El diseño siguió una metodología de normalización rigurosa para evitar redundancias y anomalías en los datos. Como se detalla en la figura 9.1, el diagrama Entidad-Relación define las bases del sistema, mientras que la figura 9.2 muestra la implementación física con sus claves foráneas y restricciones de integridad.

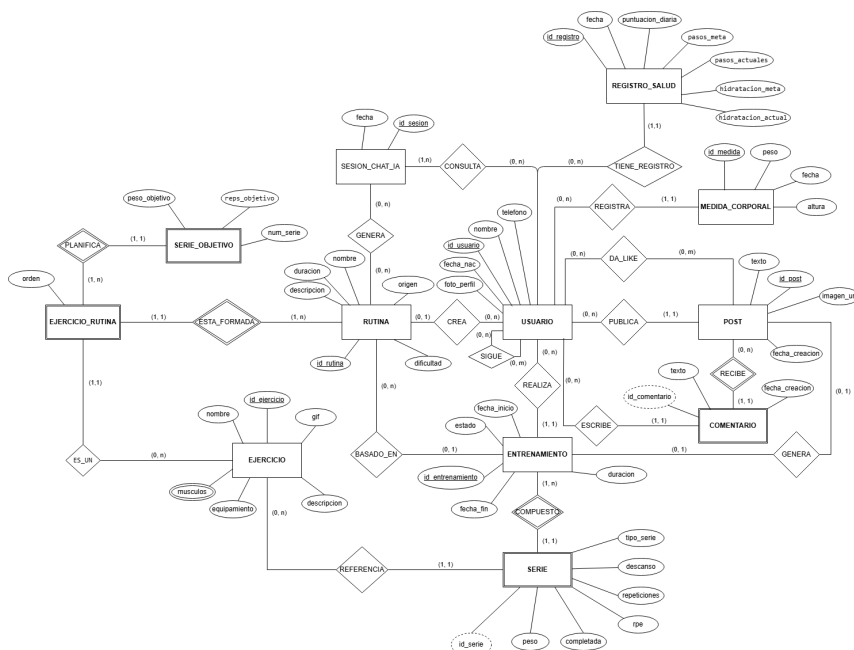


Figura 9.1: Diagrama Entidad-Relación (MER)

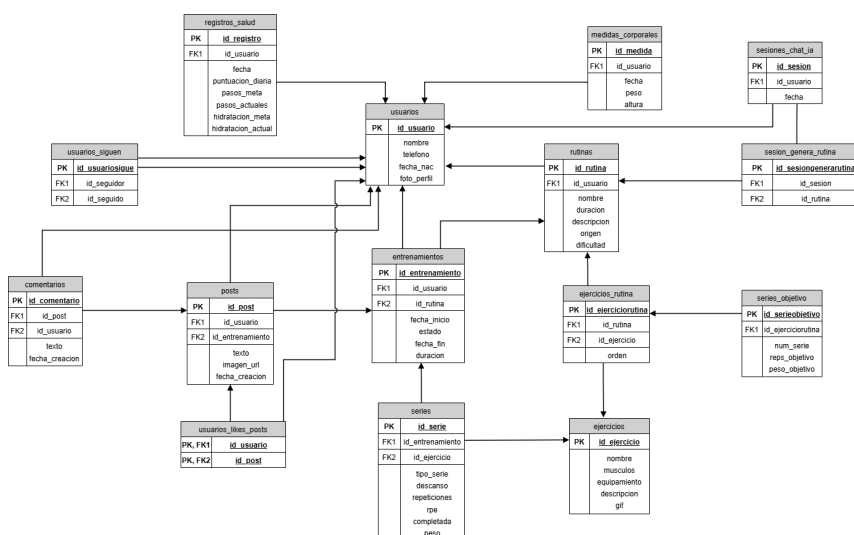


Figura 9.2: Paso a Tablas.

9.3. Interoperabilidad y Transformación de Datos

La ingesta de datos externos y la comunicación con el motor de IA requieren una capa de transformación robusta:

- **ExerciseDB Integration:** Se han importado más de 1300 ejercicios con metadatos técnicos y soporte para GIFs animados. Estos datos se normalizan e ingieren en Room durante la fase inicial de ejecución (*Pre-population*).
- **Moshi (JSON Gateway):** Actúa como el puente de datos entre las respuestas de lenguaje natural de Gemini y la base de datos relacional. La IA genera estructuras JSON que **Moshi** deserializa en objetos de datos tipados, permitiendo que el sistema realice inserciones y modificaciones autónomas en las rutinas locales sin intervención manual del usuario.
- **Health Connect:** La aplicación actúa como un integrador avanzado de la API de **Health Connect** [3]. Tras completar un entrenamiento, los datos se escriben en el repositorio compartido de Android, permitiendo que la actividad sea visible para otras aplicaciones del ecosistema.

9.4. Seguridad y Persistencia Adicional

- **Type Converters:** Utilizados para persistir tipos de datos complejos (ej: listas de pasos o sub-objetos) no soportados nativamente por SQLite.
- **Data Mapping:** Separación estricta entre las entidades de persistencia (*Entities*) y los modelos de dominio puros, garantizando que los cambios en el esquema de la DB no afecten a la lógica de negocio.
- **Jetpack DataStore:** Para ajustes de configuración y estados persistentes simples, se utiliza *DataStore* [13], asegurando la transaccionalidad y evitando bloqueos en el hilo principal.

9.5. Estrategia de Sincronización Offline-First

La fiabilidad de los datos en GymAI se basa en un paradigma *Offline-First*, donde la base de datos local actúa como la única fuente de verdad inmediata para la **UI**. El flujo lógico de decisión empleado para garantizar la consistencia entre el dispositivo y la nube se detalla en la figura 9.3:

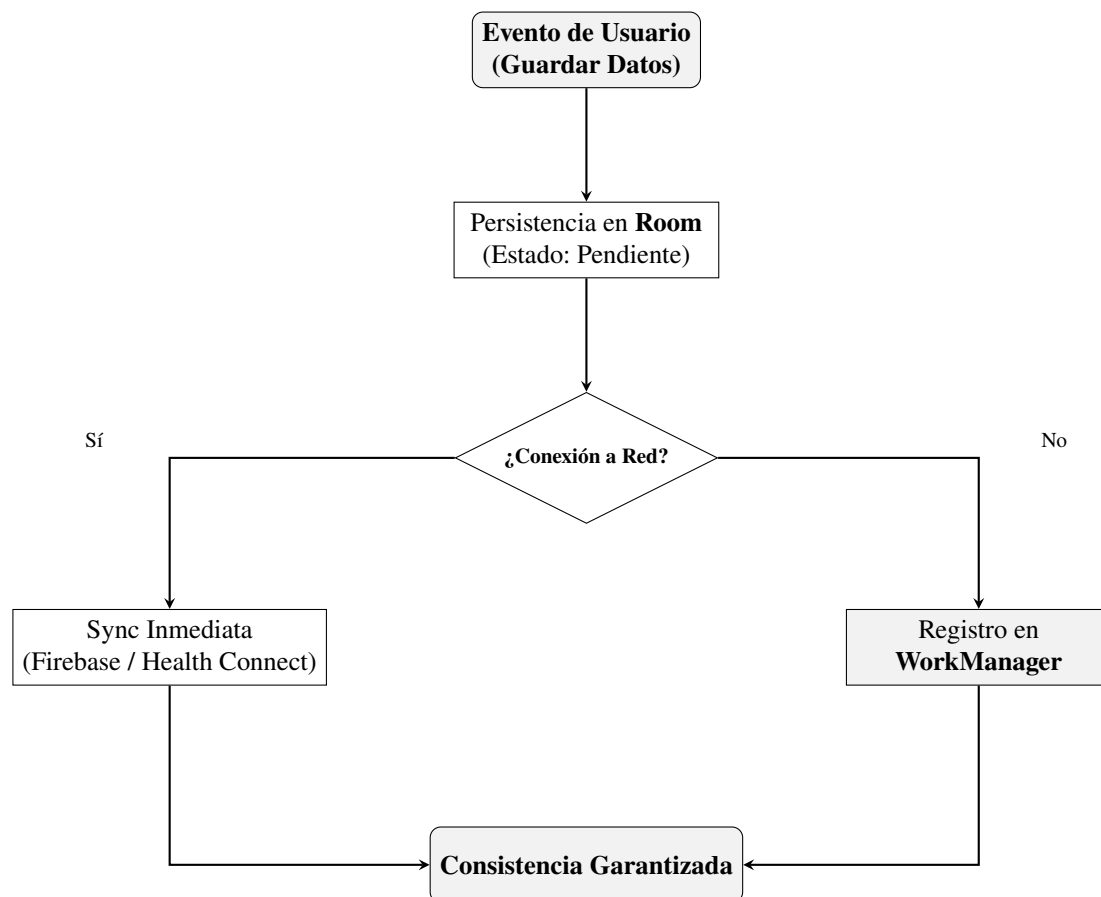


Figura 9.3: Lógica de flujo para la persistencia y sincronización de datos en GymAI.

Validación y Pruebas

Este capítulo se encuentra actualmente en fase de redacción. Próximamente detallaré aquí las estrategias de pruebas unitarias, de interfaz y de integración, así como las métricas de validación del ecosistema de agentes inteligentes.

Manual de Usuario

Este manual describe los pasos necesarios para la correcta instalación, configuración y utilización del ecosistema GymAI. El diseño de la aplicación ha sido optimizado para ofrecer una curva de aprendizaje mínima, asegurando que tanto usuarios principiantes como atletas experimentados puedan beneficiarse de la asistencia inteligente.

11.1. Requisitos Mínimos del Sistema

Para garantizar el correcto funcionamiento de GymAI, el dispositivo debe cumplir con los siguientes requisitos técnicos:

- **Sistema Operativo:** Android 11.0 (API nivel 30) o superior.
- **Hardware:** Procesador de 64 bits y un mínimo de 4GB de memoria RAM.
- **Software Adicional:** Aplicación de **Health Connect** instalada y configurada.
- **Conexión:** Acceso a internet estable para la inicialización de agentes y sincronización con Firebase.

11.2. Manual de Instalación

GymAI es un prototipo de investigación técnica. Su despliegue requiere la configuración de servicios en la nube:

1. **Configuración de Firebase:** Es necesario descargar el archivo `google-services.json` desde la consola de Firebase del desarrollador e incluirlo en el directorio `app/` del proyecto.
2. **Compilación desde Android Studio:** Clonar el repositorio, sincronizar los archivos Gradle y pulsar el botón "Run" en un dispositivo físico o emulador.

3. **Instalación mediante APK:** Si se dispone del paquete `gym_ai_release.apk`, habilitar "Orígenes desconocidos" en los ajustes de seguridad de Android.
4. **Permisos Críticos:** En la primera ejecución, el sistema solicitará acceso a **Health Connect**. Sin estos permisos, las capacidades de razonamiento biométrico de los agentes de IA estarán limitadas.

11.3. Guía de Navegación y Utilización

GymAI utiliza una interfaz moderna basada en **Material Design 3**, estructurada para minimizar la carga cognitiva del usuario.

11.4. Inicio y Autenticación

Al iniciar la aplicación, se requiere una cuenta para garantizar la persistencia de los datos entre dispositivos:

1. **Autenticación:** El sistema permite el registro mediante correo y contraseña a través de *Firebase Auth*.
2. **Sincronización de Salud:** Una vez logueado, se debe confirmar la vinculación con **Health Connect**. GymAI leerá métricas de pasos, calorías y sueño de forma automática.

11.5. Interacción con el Agent Orchestrator

El asistente inteligente (ver Figura 8.2) no es un chatbot convencional, sino un orquestador de agentes capaces de ejecutar acciones:

- **Consultas de Salud:** Puede preguntar sobre su estado actual, como "¿Cómo afecta mi falta de sueño a mi entrenamiento de hoy?". El agente de salud consultará sus métricas reales.
- **Generación de Rutinas:** Solicite planes específicos. El agente generador utilizará herramientas (*tool calling*) para devolver estructuras de datos que el usuario puede importar directamente a su biblioteca.

11.6. Ciclo de Entrenamiento y Socialización

1. **Preparación:** Seleccione una rutina de su biblioteca. Podrá ver demostraciones visuales y descripciones técnicas de cada ejercicio.

2. **Ejecución Proactiva:** Durante el entrenamiento, la app ajustará las sugerencias de carga basándose en su rendimiento en tiempo real y su historial previo.
3. **Social Feed:** Al finalizar, los logros (como nuevos récords personales) pueden compartirse en el muro social de la comunidad de GymAI.

11.7. Resiliencia y Modo Offline

La aplicación ha sido diseñada con una arquitectura **Offline-First**. Esto permite registrar entrenamientos en entornos sin cobertura. La sincronización con la nube se realiza de forma transparente en segundo plano en cuanto se recupera la conectividad, asegurando la integridad de la base de datos distribuida.

Principales Aportaciones

El desarrollo de GymAI no solo ha culminado en una herramienta funcional, sino que ha permitido validar diversas hipótesis técnicas y aportar valor en diferentes dimensiones dentro del ecosistema Android y la salud digital.

12.1. Orquestación de Agentes como Paradigma de Asistencia

La principal aportación técnica de este trabajo es la transición de una IA conversacional pasiva a un sistema de **orquestación de agentes autónomos** (ver figura 12.1).

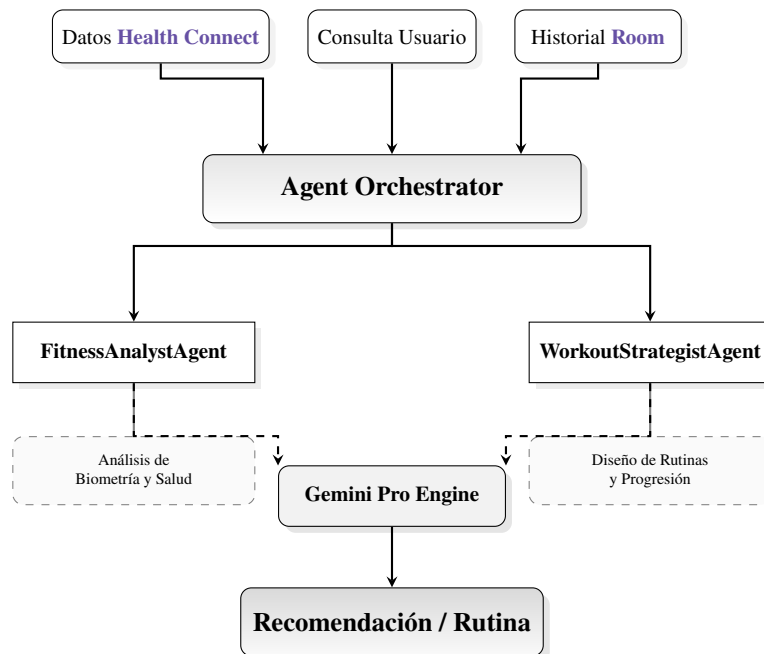


Figura 12.1: Diagrama de flujo del sistema de orquestación de agentes.

A diferencia de los asistentes basados en meros *prompts*, GymAI implementa una lógica de razonamiento capaz de:

- **Uso de Herramientas (*Tool Calling*):** Los agentes pueden invocar funciones locales de la aplicación para consultar estados biométricos o bases de datos de ejercicios.
- **Especialización de Tareas:** El sistema separa las responsabilidades entre un agente analista de salud y un estratega de rutinas, mejorando la precisión de las recomendaciones.
- **Reducción de Alucinaciones:** Al basar las respuestas en datos reales estructurados (vía *kotlinx-serialization*), se minimizan las respuestas incorrectas comunes en modelos de lenguaje generalistas.

12.2. Arquitectura Escalable y Reactiva

Se ha definido e implementado una arquitectura basada en **Clean Architecture** y **MVVM** [4] adaptada para el manejo de flujos de IA asíncronos. La integración de *Coroutines* y *Flow* garantiza que el pesado procesamiento de los modelos de Gemini no bloquee la interfaz de usuario, manteniendo una experiencia fluida incluso durante la generación de planes complejos.

12.3. Interoperabilidad Total vía Health Connect

GymAI se posiciona como un ciudadano ejemplar en el ecosistema de Google. La implementación de **Health Connect** [3] permite:

- **Agnosticismo de Hardware:** Unificación de métricas (sueño, pasos, nutrición) sin que el usuario dependa de un fabricante específico.
- **Contextualización de la IA:** Permite que el sistema orquestador utilice datos biográficos veraces para ajustar el volumen de entrenamiento de forma dinámica.

12.4. Integración con Firebase AI Logic

El uso del ecosistema oficial de **Firebase AI Logic** [24] aporta una capa de robustez y escalabilidad industrial. Este enfoque permite que la lógica de los agentes esté respaldada por la infraestructura oficial de Google, facilitando futuras expansiones del sistema y garantizando la seguridad en el manejo de las API Keys y los modelos fundacionales.

Conclusiones

Líneas de Trabajo Futuro

Aunque GymAI es una aplicación plenamente funcional en su versión actual, la naturaleza modular de su arquitectura y el uso de agentes de IA permiten abrir varias líneas de evolución que podrían transformarla en un producto comercial de referencia.

14.1. Soporte para Wear OS y Feedback Háptico

Una de las evoluciones más críticas es la posibilidad de controlar la sesión desde el pulso del deportista. El trabajo futuro incluiría:

- Desarrollar una aplicación complementaria para relojes inteligentes que replique el estado de la sesión activa.
- Sincronización en tiempo real de la frecuencia cardíaca para que el ecosistema de agentes pueda ajustar los tiempos de descanso basándose en la recuperación cardiovascular real del usuario.

14.2. Visión por Computador para Corrección Técnica

La integración de modelos de *Pose Estimation* (utilizando *ML Kit*) permitiría que la cámara del dispositivo analizara en tiempo real la forma del ejercicio:

- Detectar errores posturales críticos (como el redondeo de la espalda en el peso muerto) y proporcionar correcciones auditivas mediante el agente.
- Registro automático de repeticiones por visión computacional, eliminando la fricción de la interacción táctil durante el esfuerzo.

14.3. Inteligencia Nutricional Multimodal

El asistente de agentes podría expandirse hacia un módulo nutricional avanzado que:

- Analice imágenes de platos de comida para estimar macronutrientes de forma autónoma.
- Cruce el gasto calórico metabólico (obtenido de Health Connect) con la ingesta estimada para sugerir ajustes dinámicos diarios en los objetivos energéticos.

14.4. Privacidad y "On-Device AI" con Gemini Nano y Gemma

Para lograr una independencia total de la infraestructura en la nube y garantizar el máximo nivel de privacidad, se propone la migración hacia modelos de lenguaje locales (On-device LLMs):

- **Integración con AICore y Gemini Nano:** Uso de la infraestructura nativa de Android para ejecutar tareas de razonamiento local directo sobre el hardware (NPU). Esto eliminaría la latencia de red y los costes operativos de APIs externas.
- **Familia Gemma de Google:** Evaluación de modelos abiertos como *Gemma 2B* para tareas especializadas que requieran una personalización (*fine-tuning*) fuera del motor nativo de AICore.
- **RGPD y Soberanía de Datos:** Al procesar la biometría sensible íntegramente en el dispositivo, se garantiza que la información de salud nunca se transmita a servidores de terceros, cumpliendo así con las normativas más estrictas de protección de datos personales.

14.5. Sincronización con Gimnasios Inteligentes (IoT)

La eliminación de la fricción en la toma de datos es un paso natural para la evolución de GymAI:

- **Protocolos de Conectividad:** Implementación de *Bluetooth Low Energy* (BLE) y *NFC* para la comunicación directa con equipamiento de gimnasio inteligente.
- **Automatización del Registro:** Captura automática de cargas, repeticiones y velocidad de ejecución (*Velocity Based Training - VBT*) mediante la integración con los SDKs de fabricantes líderes en maquinaria de fitness, eliminando la necesidad de entrada manual por parte del usuario.

14.6. Orquestación de Enjambres de Agentes (Agent Swarms)

La arquitectura actual basada en dos agentes puede evolucionar hacia una estructura jerárquica de expertos:

- **Colaboración Multi-Agente:** Implementación de un "Enjambre de Agentes" donde perfiles especializados (Fisioterapeuta, Nutricionista y Coach de Fuerza) debaten y validan el plan de entrenamiento en segundo plano antes de presentarlo al usuario.
- **Resolución de Conflictos Biológicos:** Este enfoque permitiría resolver situaciones complejas donde, por ejemplo, el agente Coach propone una alta intensidad, pero el agente Fisioterapeuta veta la decisión basándose en una leve inflamación detectada en las métricas de variabilidad de la frecuencia cardíaca (HRV).

14.7. Gamificación y Ligas Comunitarias

Para aumentar la retención y la adherencia al entrenamiento, se podrían implementar desafíos comunitarios avanzados:

- Ligas por categorías de fuerza relativa y progresión.
- Sistema de "Marketplace de Rutinas Verificadas" donde los usuarios puedan compartir planes de entrenamiento evaluados por el ecosistema de IA.

Referencias

- [1] R. Meeusen et al., «Prevention, diagnosis, and treatment of the overtraining syndrome,» *Medicine & Science in Sports & Exercise*, vol. 45, n.º 1, págs. 186-205, 2013.
- [2] Google. «Gemini 2.5 Flash Technical Documentation. »Disponible en: <https://ai.google.dev/models/gemini>
- [3] Google. «Health Connect API documentation. »Disponible en: <https://developer.android.com/health-and-fitness/guides/health-connect>
- [4] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017, ISBN: 978-0134494166.
- [5] Google. «Guide to app architecture. »Disponible en: <https://developer.android.com/topic/architecture>
- [6] Google, *Dependency injection with Hilt*, <https://developer.android.com/training/dependency-injection/hilt-android>, 2024.
- [7] Google, *Test apps on Android*, <https://developer.android.com/training/testing>, 2024.
- [8] Google. «Save data in a local database using Room. »Disponible en: <https://developer.android.com/training/data-storage/room>
- [9] RapidAPI. «ExerciseDB API Documentation. »Disponible en: <https://rapidapi.com/justin-m-1/api/exercisedb>
- [10] K. Schwaber y J. Sutherland, *The Scrum Guide*, <https://scrumguides.org/scrum-guide.html>, 2020.
- [11] Figma, *Figma Help Center*, <https://help.figma.com/>, 2024.
- [12] Google, *Stitch: Material Design Tool*, <https://stitch.withgoogle.com/>, 2024.
- [13] Google, *Jetpack DataStore*, <https://developer.android.com/topic/libraries/architecture/datastore>, 2024.
- [14] Google, *Firebase Authentication*, <https://firebase.google.com/docs/auth>, 2024.

- [15] Square, Inc., *Moshi: A modern JSON library for Android and Java*, <https://github.com/square/moshi>, 2024.
- [16] JetBrains. «Kapt: Kotlin Annotation Processing Tool. »Disponible en: <https://kotlinlang.org/docs/kapt.html>
- [17] Google. «Kotlin Symbol Processing (KSP). »Disponible en: <https://developer.android.com/build/migrate-to-ksp>
- [18] Google. «Jetpack Compose Documentation. »Disponible en: <https://developer.android.com/develop/ui/compose>
- [19] Google, *Modern Android Development*, <https://developer.android.com/modern-android-development>, 2024.
- [20] JetBrains, *Coroutines guide*, <https://kotlinlang.org/docs/coroutines-guide.html>, 2024.
- [21] JetBrains, *Asynchronous Flow*, <https://kotlinlang.org/docs/flow.html>, 2024.
- [22] Google, *Material Design 3 Guidelines*, <https://m3.material.io/>, 2024.
- [23] Coil Contributors, *Coil: Image loading for Android*, <https://coil-kt.github.io/coil/>, 2024.
- [24] Google. «Firebase AI Logic Documentation. »Disponible en: <https://firebase.google.com/docs/ai>
- [25] Antigravity Team, *Antigravity: Intelligent AI Agent*, Software de asistencia, 2026.
- [26] Google, *Android Studio Official IDE*, <https://developer.android.com/studio/intro>, 2024.
- [27] GitHub, *GitHub Documentation*, <https://docs.github.com/>, 2024.
- [28] Gemini Team, Google, «Gemini 2.5: Unlocking multimodal understanding across millions of tokens of context,» arXiv, inf. téc. arXiv:2403.05530, 2024, <https://arxiv.org/abs/2403.05530>.
- [29] JetBrains. «Kotlinx.serialization guide. »Disponible en: <https://github.com/Kotlin/kotlinx.serialization>
- [30] Square, Inc., *Retrofit: Type-safe HTTP client*, <https://square.github.io/retrofit/>, 2024.
- [31] JUnit Team, *JUnit 5 User Guide*, <https://junit.org/junit5/docs/current/user-guide/>, 2024.
- [32] Oleksii Raikov, *MockK: mocking library*, <https://mockk.io/>, 2024.
- [33] Hevy Team, *Hevy - Gym Log Tracker*, <https://www.hevyapp.com/>, 2024.

Anexos

A.1. Viabilidad Económica

Para evaluar la sostenibilidad del proyecto, se ha realizado un estudio de los costes operativos estimados para el primer año de operación de **GymAI** en un entorno de producción real. Esta estimación se centra en los gastos directos de infraestructura cloud y servicios de inteligencia artificial necesarios para mantener el sistema plenamente funcional bajo demanda.

Concepto	Coste Unitario	Frecuencia	Subtotal
Infraestructura Cloud (Firebase Blaze)	30,00 €	12 meses	360,00 €
IA Cognitiva (Gemini 2.5 API)	50,00 €	12 meses	600,00 €
API de Ejercicios (ExerciseDB - Pro)	13,00 €	12 meses	156,00 €
Licencia Google Play Console	25,00 €	Pago único	25,00 €
TOTAL ESTIMADO (Primer año)			1.141,00 €

Cuadro A.1: Estimación de costes operativos para el mantenimiento de GymAI.

Los datos presentados confirman que el mantenimiento técnico del prototipo es altamente sostenible, permitiendo la validación del modelo de negocio con una inversión mínima en servicios cloud y procesamiento de IA a escala.

A.2. Resumen Estratégico: Lean Canvas

Siguiendo metodologías ágiles de negocio, se presenta a continuación el **Lean Canvas** de **GymAI**. Este modelo permite visualizar de forma rápida la propuesta de valor única del sistema y su encaje en el mercado actual del fitness digital.

1. Problema - Falta de acceso a entrenamiento personal. - Datos sin interpretar. - Pérdida de motivación.	2. Solución - Agente de IA proactivo. - Integración biométrica. - Privacidad (On-device).	3. Propuesta de Valor Única Entrenadores personales inteligentes 24/7, que entienden tu fatiga y evolucionan contigo gracias a la IA.
4. Ventaja Competitiva - On-device AI (Gemini Nano). - Arquitectura Swarm de agentes.	5. Segmentos de Clientes - Entusiastas del fitness. - Usuarios que valoran su privacidad.	6. Canales - Google Play Store. - Redes Sociales (Fitness).
7. Métricas Clave - Engagement diario (DAU). - Tasa de mejora física.	8. Estructura de Costes - Desarrollo (CAPEX). - Cloud y API (OPEX).	9. Flujos de Ingresos - Freemium (Agente Pro). - Afiliación inteligente.

Cuadro A.2: Lean Canvas del proyecto GymAI.

A.3. Capturas del Prototipo en Figma

El diseño original y el flujo de navegación interactivo pueden consultarse en el siguiente enlace: <https://www.figma.com/proto/Zvmn0sU2vQ4iE79cxBXoXz/GymAI>.

A.4. Lista de Acrónimos

A continuación se detalla el significado de las siglas y términos abreviados empleados a lo largo de este Trabajo de Fin de Grado. Todos los términos son interactivos y permiten volver a la página de su primera aparición:

API	Interfaz de Programación de Aplicaciones	8, 14 s.
APK	Android Package Kit	12
GENAI	Inteligencia Artificial Generativa	3
HU	Historia de Usuario	9, 21

IA Inteligencia Artificial	1–7, 9 ss., 14
JSON JavaScript Object Notation	3, 12, 20
LLM Modelos de Lenguaje de Gran Escala	2
MAD Modern Android Development	18
MER Modelo Entidad-Relación	11
MVVM Modelo-Vista-Vista Modelo	4, 14, 38
SSOT Fuente Única de Verdad	15
TFG Trabajo de Fin de Grado	3
UI Interfaz de Usuario	14 s., 19, 32

A.5. Glosario de Términos

Para facilitar la comprensión técnica del documento, se definen a continuación los conceptos clave utilizados en el desarrollo del sistema:

Agile Metodología de desarrollo de software basada en iteraciones rápidas y entregas continuas de valor	9 s.
Android Sistema operativo móvil basado en el núcleo Linux y otros softwares de código abierto6, 15, 18	
Antigravity Asistente de inteligencia artificial integrado en el IDE para la generación de código y validación técnica	20
Chatbot Programa informático diseñado para simular conversaciones con usuarios humanos, especialmente a través de internet.	12
Clean Architecture Patrón arquitectónico que promueve la separación de responsabilidades y la independencia de los componentes	9, 11, 14, 38
DAO Data Access Object. Patrón que abstrae y encapsula todo el acceso a la fuente de datos11, 15	
DataStore Solución de almacenamiento de datos de Jetpack que permite guardar pares clave-valor o objetos tipados	11
ExerciseDB API externa que proporciona un catálogo extenso de ejercicios de fitness categorizados por equipo y grupo muscular	8, 11, 15

Feature Branching	Modelo de desarrollo basado en la creación de ramas independientes para cada nueva funcionalidad o corrección	11
Figma	Herramienta de edición de gráficos vectorial y generación de prototipos, principalmente basada en la web	11, 19
Firebase	Plataforma de desarrollo de aplicaciones móviles de Google que proporciona herramientas para autenticación, bases de datos y análisis	5
Firebase AI Logic	Infraestructura para ejecutar lógica de IA mediante Cloud Functions .	2, 4, 38
Firebase Auth	Servicio de autenticación que proporciona SDKs y bibliotecas para gestionar usuarios en la aplicación	12
Firestore	Base de datos NoSQL flexible y escalable para el desarrollo para móviles de Firebase	12, 15
Flow	Estructura de streams reactivos asíncronos construida sobre corrutinas de Kotlin	19
Function Calling	Capacidad de los modelos de lenguaje para estructurar salidas JSON que permiten la ejecución de funciones externas predefinidas	15
Gemini	Familia de modelos de lenguaje de gran escala desarrollados por Google DeepMind	2, 5, 11, 15
GitHub	Plataforma de desarrollo colaborativo basada en la nube que utiliza el sistema de control de versiones Git	11
Google Stitch	Herramienta de diseño que permite sincronizar tokens de estilo y componentes entre Figma y el código del proyecto móvil	11, 19
Gradle	Sistema de automatización de compilación de código abierto centrado en la flexibilidad y el rendimiento	11
Health Connect	Repositorio de datos de salud de Google para Android	2 ss., 6 s., 12, 15, 29, 34 s., 38
Hilt	Librería de inyección de dependencias para Android que simplifica el uso de Dagger en aplicaciones nativas	5, 11, 16, 18
Jetpack Compose	Kit de herramientas de Google para la creación de interfaces nativas declarativas en Android	2, 5, 14, 18 s., 26
K2	Nueva generación del compilador de Kotlin que ofrece mejoras significativas en rendimiento y análisis de código	19
Kapt	Kotlin Annotation Processing Tool. Herramienta para el procesamiento de anotaciones en Kotlin que permite la generación de código durante la compilación	12

Kotlin	Lenguaje de programación moderno, multiplataforma y de tipado estático, utilizado principalmente para el desarrollo de aplicaciones Android.....	2, 18 s.
Kotlin Coroutines	Librería de Kotlin para el desarrollo de programación asíncrona no bloqueante de forma secuencial	19
KSP	Kotlin Symbol Processing. Herramienta nativa de Kotlin para la generación de código y análisis de símbolos durante la compilación	12
Moshi	Librería moderna de JSON para Android y Java que facilita la conversión de JSON a objetos tipados.....	12
Offline-First	Diseño que garantiza el funcionamiento local sin conexión, sincronizando datos al recuperar la red.....	2, 5, 8, 12, 36
Product Backlog	Lista priorizada de todas las funcionalidades, requisitos y correcciones necesarias en el producto	10
Prompt Engineering	Proceso de estructurar y refinar las instrucciones de texto para optimizar el rendimiento de los modelos de lenguaje	11
Repository	Patrón de diseño que actúa como mediador entre las fuentes de datos y la lógica de negocio, abstrayendo el origen de la información	15
Room	Librería de persistencia de Android que proporciona una capa de abstracción sobre SQLite para un acceso a datos fluido	7, 11 s., 14
Scrum	Marco de trabajo ágil para la gestión del desarrollo de productos complejos	10
Sprint	Bloque de tiempo fijo en el que se completa un incremento funcional del producto	10
SQLite	Motor de base de datos relacional ligero e integrado que Room utiliza como almacenamiento persistente local	15
StateFlow	Emisor de flujo reactivo que mantiene el estado actual de los datos y emite actualizaciones a los observadores activos	15
ViewModel	Componente encargado de gestionar el estado de la UI y la comunicación con la capa de dominio, manteniendo la lógica de presentación desacoplada	14
WorkManager	Librería de Android para programar tareas en segundo plano que deben ejecutarse de manera persistente	12